

10

Appendix

Well, my dear reader, we have reached the last part of this book: the summary. I sincerely hope you enjoyed reading what I had to say about state management libraries in **React Native** and I want to thank you for getting this far. Let me take you now on a trip down memory lane of everything we talked about in this book. And if afterward you're not too tired of my thoughts and ruminations, you will find a bonus section on recruitment interview questions related to **state management**.

In the first chapters of this book, we looked very broadly at the history of web development. We saw the evolution of the internet landscape, which led to the creation of **ReactJS**. Then, we talked about the evolution of **React** itself, which led to the creation of React Native. Knowing how close React Native is to ReactJS can be of great help while working on React Native apps. The ReactJS community is bigger and more mature than

its mobile-first cousin. Many issues that React Native developers face can be solved with ReactJS knowledge. There's a notion called the **React mindset**, which is crucial for writing robust, scalable, and bug-free apps. There are many great articles on this topic, for example, the *Thinking in React* article posted inside the official React documentation. Once we learned how to adopt this mindset, we started building our very own app: Funbook.

Unsurprisingly, the app we created is a social media clone app. Social media apps are an interesting topic for example code, as most of us are very familiar with how they should work. At the same time, they are much more complicated than the traditional to-do app, present in most ReactJS tutorials. Setting up any mobile app is a task on its own. For all the web developers out there, working on mobile apps is new territory, with its own tooling and processes. Luckily, we can take advantage of **Expo**, and have a functioning and testable app in minutes. Once we got comfortable with the basic app setup, we got to coding the real Funbook app. We added a few surfaces: Feed, Conversations, Liked Images, and Camera. Then we got to think in React! We planned and wrote the underlying components of all the surfaces. We used many modern React features, such as hooks and context. By the end of ***Chapter 4***, we had a beautiful, functioning mobile app, which we could test on real devices or on phone simulators on our computer screens. This

may seem like a lot of work, but let me assure you: before React Native, and a few of its **JavaScript** predecessors, creating mobile apps working on **Android** and **iOS** was a lot more work!

Chapter 5, *Implementing Redux in Our Funbook App*, was the first to talk about external solutions for state management in React Native apps. The specific solution we talked about was **Redux** and **Redux Toolkit**. Redux is the oldest and most widely known and used state management library in the React community, as of the time of writing this book. When used wisely, it's a great tool. It requires quite a bit of boilerplate, and its creator has doubts about how it's implemented. However, the team behind Redux Toolkit has made great progress in keeping this library developer-friendly and up to date. We configured Redux and Redux Toolkit in the Funbook app, and we saw how to use them for managing the list of liked images.

In the next chapter, we talked about a library considered to be the second most popular in the React community: **MobX**. By this time, we were armed with a sound knowledge of ReactJS, React Native, and some thoughts on how global state could be managed by React alone or with Redux. MobX invited us to rethink a few preconceptions and look at global state management differently. Instead of passing props or actions through an intricate web of components, MobX gives us tools to use

global state data as any other prop, while only informing components about being observed. We learned later that this sort of global state management is sometimes called **proxy-based**. The state management library stands between the user and the code, managing state in a sort of invisible layer, like a proxy on the web. MobX is sometimes compared to **Valtio**, another proxy-based state management library.

After learning about MobX observables, actions, and their approach to deriving state values (which should be done as much as possible), we were ready to use it. We implemented the same functionality as for Redux – managing the list of liked images. And once we had that working in MobX, we moved on to the next state management library: **XState**.

Xstate is less popular than Redux and MobX, but it offers yet another way of looking at global state management. And even better, it offers a dedicated tool for doing so! The **Xstate visualizer** is an incredible tool that can be used for working in any global state for any app. Being able to see how different pieces of state relate to each other can come in handy when you're tasked with creating a new app. Xstate not only provides this great tool, but its creator invites us also to take a more mathematical approach to state management. Thanks to him, we can learn what a state machine is and that every part of the global state in an app should always be in a defined state.

After playing around with Xstate, and of course, implementing the list of liked images with it, we were ready to move on. The next library we looked at was **Jotai**.

Jotai was considered the new kid on the block when I started writing this book. That was many moons ago! As of the time of writing this summary, there are a few newer state management libraries. I feared they were not mature enough to be analyzed along with big players such as Redux and MobX. Jotai, however, has been holding strong over the last few months and getting more and more attention from the community. Jotai was heavily inspired by **Recoil**, a state management library created by the React team at Meta. Recoil remains in an experimental state, but Jotai is ready for usage in production apps. The main concept in Jotai is the use of **atoms**. They are the smallest building block of global state that we can pepper around the app – just like we could pepper it with **useState** hooks. The big difference is that Jotai's atoms will be freely available throughout the app, without unpleasant prop drilling or extensive boilerplate. Using Jotai for the list of liked images felt, for me, a little bit magical: a minimal configuration and we can access pieces of state wherever we'd like!

Once we used Jotai in the Funbook app, we were ready to abandon it and move on to the next thing. And the next thing was very different from its predecessors – **React Query**, and

the notion that we may not need any state management library at all. React Query is not a state management library; it's a library created for better data management and synchronization between an app and the server. It aims at reducing net-



documentation is exhaustive, and it is accompanied by a specialized blog. Dozens of common developer problems are solved within the library itself. We used React Query, or **TanStack Query**, for fetching the list of liked images. We were, unfortunately, unable to use other features it offers, such as data mutations, as the backend of the Funbook app is quite minimal.

The creators of React Query ask a very good question: do you really need a state management library for your app? Let's ask ourselves this same question. We were able to create the Funbook app using React alone. We were also able to use React Query mixed with local state. Does this mean all dedicated state management libraries should be wiped from the earth along with this book? Certainly not.

Choosing a state management library, when choosing from battle-tested solutions, boils down to developer experience. The end user of your app will not know whether you're using Jotai or Redux, but your fellow developers may give you an earful about it. Some developers live and breathe Redux, while others

would rather not touch Redux-based projects. There is a silent global consensus in the community that state management libraries should not be used for fetching and persisting data in apps. This task should be left for better-suited libraries, such as React Query. So, maybe the next app you create will use MobX for local state and React Query for fetching data? Or maybe Xstate for local state, **Axios** for fetching, and **Async Storage** for persisting state? Or maybe something else completely. I believe every state management library has its strong points, as well as weaknesses. I also believe discussing which is better is a moot point, as neither is objectively better. I hope that thanks to this book, you were able to “dip your toe” in a few different solutions and you are more aware of what you personally prefer. And once you find what you like, have fun working with it!

Bonus content

Speaking of work: you may find yourself, my dear reader, going through job interviews where you are asked about React, React Native, and state management solutions. There are a few questions I have gotten myself that I found were either very common or very interesting. I’ve compiled a list of those questions in the hope that they facilitate your next recruitment. Questions regarding React and Redux pop up in most job interviews for roles related to software development with React and React Native. Questions about other state management li-

libraries may get asked if you specify that you are familiar with the given libraries. To be honest, 80% of job offers list React and Redux. I'm hoping this will change in the coming months and years, as other state management libraries offer great solutions. Here are some common or interesting questions:

1. In React, what is the difference between **props** and **state**?
2. Is it necessary to use an external state management library in a React Native app?
3. In Redux, what is a reducer and an action?
4. In Redux, what is the advantage of using selectors?
5. In Redux, are you allowed to change state values directly?
6. In MobX, what is a model?
7. In MobX, how do you make a component aware of the global state values?
8. In Xstate, what is a state machine?
9. In Xstate, how do you pass additional data through the state machine?
10. In Jotai, what is the name of the most basic piece of state?
11. Can you replace all state management with React Query alone?

I'm giving you only the questions because giving you the answers would be a tad too easy, don't you think? If you must go back in the book and research the answers, or maybe simply

google them, there's a better chance the information will stick with you.

I sincerely hope you enjoyed reading this book just as much as I enjoyed writing it! Thank you for sticking around, and feel free to contact me through Twitter (if it still exists by the time this book gets published!). Good night and good luck!

